IT草根

Lucene 6.0 实战（5）–索引搜索器IndexSearcher

2016-05-24 | 2017-11-20 | Programming Notes | 1680

Lucene的主要搜索API

一个简单的搜索应用主要包括索引和搜索两部分，在Lucene中，IndexSearcher类是用于对索引中文档进行搜索的核心类，它有几个重载的搜索方法，可以使用最常用的方法对特定的项进行搜索，一个项由一个字符串类型的域值和对应的域名构成。现将搜索相关API汇总如下

类	目的
IndexSearcher	搜索索引的核心类。所有搜索都通过IndexSearcher进行，它们会调用该类中重载的search方法
Query及其子类	封装某种查询类型的具体子类。Query实例将被传递给IndexSearcher的search方法
QueryParser	将用户输入的可读的查询表达式处理成具体的Query对象
TopDocs	保持由IndexSearcher.search()方法返回的具有较高评分的顶部文档
ScoreDoc	提供对TopDocs中每条搜索结果的访问接口

IndexSearcher简介

首先看IndexSearcher的doc注释

> Implements search over a single IndexReader.
> Applications usually need only call the inherited search(Query, int) or search(Query, Filter, int) methods. For performance reasons, if your index is unchanging, you should share a single IndexSearcher instance across multiple searches instead of creating a new one per-search. If your index has changed and you wish to see the changes reflected in searching, you should use DirectoryReader.openIfChanged(DirectoryReader) to obtain a new reader and then create a new IndexSearcher from that. Also, for low-latency turnaround it’s best to use a near-real-time reader (DirectoryReader.open(IndexWriter)). Once you have a new IndexReader, it’s relatively cheap to create a new IndexSearcher from it.
> NOTE: IndexSearcher instances are completely thread safe, meaning multiple threads can call any of its methods, concurrently. If your application requires external synchronization, you should not synchronize on the IndexSearcher instance; use your own (non-Lucene) objects instead.

猜你喜欢

从这段说明中得出如下几点

- 提供了对单个IndexReader的查询实现
- 通常应用程序只需要调用search(Query, int)或者search(Query, Filter, int)方法
- 如果你的索引不变，在多个搜索中应该采用共享一个IndexSearcher实例的方式
- 如果索引有变动，并且你希望在搜索中有所体现，那么应该使用DirectoryReader.openIfChanged(DirectoryReader)来获取新的reader，然后通过这个reader创建一个新的IndexSearcher

- 为了低延迟查询，最好使用近实时搜索（NRT），此时构建IndexSearcher需要使用DirectoryReader.open(IndexWriter)，一旦你获取一个新的IndexReader，再去创建一个IndexSearcher所付出的代价要小的多
- IndexSearcher实例是完全线程安全的，这意味着多个线程可以并发调用任何方法。如果需要外部同步，无需对IndexSearcher实例进行同步

Lucene的多样化查询

- 通过项进行搜索–TermQuery类
- 在指定的项范围内搜索–TermRangeQuery类
- 通过字符串搜索–PrefixQuery类
- 组合查询–BooleanQuery类
- 通过短语搜索–PhraseQuery类
- 通配符查询–WildcardQuery类
- 搜索类似项–FuzzyQuery类
- 匹配所有文档–MatchAllDocsQuery类
- 不匹配文档–MatchNoDocsQuery类
- 解析查询表达式–QueryParser类
- 多短语查询–MultiPhraseQuery类

一些简单的查询示例如下

```
import org.apache.lucene.analysis.Analyzer;
import org.apache.lucene.analysis.TokenStream;
import org.apache.lucene.analysis.core.WhitespaceAnalyzer;
import org.apache.lucene.analysis.standard.StandardAnalyzer;
import org.apache.lucene.analysis.tokenattributes.OffsetAttribute;
import org.apache.lucene.document.Document;
import org.apache.lucene.document.Field;
import org.apache.lucene.document.StringField;
import org.apache.lucene.document.TextField;
import org.apache.lucene.index.DirectoryReader;
import org.apache.lucene.index.IndexWriter;
import org.apache.lucene.index.IndexWriterConfig;
import org.apache.lucene.index.Term;
import org.apache.lucene.queryparser.classic.ParseException;
import org.apache.lucene.queryparser.classic.QueryParser;
import org.apache.lucene.search.*;
import org.apache.lucene.store.Directory;
import org.apache.lucene.store.RAMDirectory;
import org.apache.lucene.util.BytesRef;
import org.junit.Assert;
import org.junit.Before;
import org.junit.Test;

import java.io.IOException;
import java.io.StringReader;

public class IndexSearchDemo {
    private Directory directory = new RAMDirectory();
    private String[] ids = {"1", "2"};
    private String[] countries = {"Netherlands", "Italy"};
    private String[] contents = {"Amsterdam has lots of bridges", "Venice has lots of canals, not bridg
    private String[] cities = {"Amsterdam", "Venice"};
    private IndexWriterConfig indexWriterConfig = new IndexWriterConfig(new WhitespaceAnalyzer());
    private IndexWriter indexWriter;

    @Before
    public void createIndex() {
        try {
            indexWriter = new IndexWriter(directory, indexWriterConfig);
            for (int i = 0; i < 2; i++) {
                Document document = new Document();
```

```
                Field idField = new StringField("id", ids[i], Field.Store.YES);
                Field countryField = new StringField("country", countries[i], Field.Store.YES);
                Field contentField = new TextField("content", contents[i], Field.Store.NO);
                Field cityField = new StringField("city", cities[i], Field.Store.YES);
                document.add(idField);
                document.add(countryField);
                document.add(contentField);
                document.add(cityField);
                indexWriter.addDocument(document);
            }
            indexWriter.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    @Test
    public void testTermQuery() throws IOException {
        Term term = new Term("id", "2");
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(new TermQuery(term), 10);
        Assert.assertEquals(1, search.totalHits);
    }

    @Test
    public void testMatchNoDocsQuery() throws IOException {
        Query query = new MatchNoDocsQuery();
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(query, 10);
        Assert.assertEquals(0, search.totalHits);
    }

    @Test
    public void testTermRangeQuery() throws IOException {
        //搜索起始字母范围从A到Z的city
        Query query = new TermRangeQuery("city", new BytesRef("A"), new BytesRef("Z"), true, true);
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(query, 10);
        Assert.assertEquals(2, search.totalHits);
    }

    @Test
    public void testQueryParser() throws ParseException, IOException {
        //使用WhitespaceAnalyzer分析器不会忽略大小写，也就是说大小写敏感
        QueryParser queryParser = new QueryParser("content", new WhitespaceAnalyzer());
        Query query = queryParser.parse("+lots +has");
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(query, 1);
        Assert.assertEquals(2, search.totalHits);
        query = queryParser.parse("lots OR bridges");
        search = indexSearcher.search(query, 10);
        Assert.assertEquals(2, search.totalHits);

        //有点需要注意，在QueryParser解析通配符表达式的时候，一定要用引号包起来，然后作为字符串传递给parse函数
        query = new QueryParser("field", new StandardAnalyzer()).parse("\"This is some phrase*\"");
        Assert.assertEquals("analyzed", "\"? ? some phrase\"", query.toString("field"));

        //语法参考：http://lucene.apache.org/core/6_0_0/queryparser/org/apache/lucene/queryparser/classic
        //使用QueryParser解析"~"，~代表编辑距离，~后面参数的取值在0-2之间，默认值是2，不要使用浮点数
        QueryParser parser = new QueryParser("city", new WhitespaceAnalyzer());
        //例如，roam~，该查询会匹配foam和roams，如果~后不跟参数，则默认值是2
        //QueryParser在解析的时候不区分大小写（会全部转成小写字母），所以虽少了一个字母，但是首字母被解析为小写的v，依然
        query = parser.parse("Venic~2");
        search = indexSearcher.search(query, 10);
        Assert.assertEquals(1, search.totalHits);
```

```
    }

    @Test
    public void testBooleanQuery() throws IOException {
        Query termQuery = new TermQuery(new Term("country", "Beijing"));
        Query termQuery1 = new TermQuery(new Term("city", "Venice"));
        //测试OR查询，或者出现Beijing或者出现Venice
        BooleanQuery build = new BooleanQuery.Builder().add(termQuery, BooleanClause.Occur.SHOULD).add(
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(build, 10);
        Assert.assertEquals(1, search.totalHits);
        //使用BooleanQuery实现 国家是(Italy OR Netherlands) AND contents中包含(Amsterdam)操作
        BooleanQuery build1 = new BooleanQuery.Builder().add(new TermQuery(new Term("country", "Italy")
                (new Term("country",
                        "Netherlands")), BooleanClause.Occur.SHOULD).build();
        BooleanQuery build2 = new BooleanQuery.Builder().add(build1, BooleanClause.Occur.MUST).add(new
                .MUST).build();
        search = indexSearcher.search(build2, 10);
        Assert.assertEquals(1, search.totalHits);
    }

    @Test
    public void testPhraseQuery() throws IOException {
        //设置两个短语之间的跨度为2，也就是说has和bridges之间的短语小于等于均可检索到
        PhraseQuery build = new PhraseQuery.Builder().setSlop(2).add(new Term("content", "has")).add(ne
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(build, 10);
        Assert.assertEquals(1, search.totalHits);
        build = new PhraseQuery.Builder().setSlop(1).add(new Term("content", "Venice")).add(new Term("c
                "canals")).build();
        search = indexSearcher.search(build, 10);
        Assert.assertNotEquals(1, search.totalHits);
    }

    @Test
    public void testMatchAllDocsQuery() throws IOException {
        Query query = new MatchAllDocsQuery();
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(query, 10);
        Assert.assertEquals(2, search.totalHits);
    }

    @Test
    public void testFuzzyQuery() throws IOException, ParseException {
        //注意是区分大小写的，同时默认的编辑距离的值是2
        //注意两个Term之间的编辑距离必须小于两者中最小者的长度：the edit distance between the terms must be less
        Query query = new FuzzyQuery(new Term("city", "Veni"));
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(query, 10);
        Assert.assertEquals(1, search.totalHits);
    }

    @Test
    public void testWildcardQuery() throws IOException {
        //*代表0个或者多个字母
        Query query = new WildcardQuery(new Term("content", "*dam"));
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(query, 10);
        Assert.assertEquals(1, search.totalHits);
        //?代表0个或者1个字母
        query = new WildcardQuery(new Term("content", "?ridges"));
        search = indexSearcher.search(query, 10);
        Assert.assertEquals(2, search.totalHits);
        query = new WildcardQuery(new Term("content", "b*s"));
        search = indexSearcher.search(query, 10);
```

```
        Assert.assertEquals(2, search.totalHits);
    }

    @Test
    public void testPrefixQuery() throws IOException {
        //使用前缀搜索以It打头的国家名，显然只有Italy符合
        PrefixQuery query = new PrefixQuery(new Term("country", "It"));
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(query, 10);
        Assert.assertEquals(1, search.totalHits);
    }

    private IndexSearcher getIndexSearcher() throws IOException {
        return new IndexSearcher(DirectoryReader.open(directory));
    }

    @Test
    public void testToken() throws IOException {
        Analyzer analyzer = new StandardAnalyzer();
        TokenStream
                tokenStream = analyzer.tokenStream("myfield", new StringReader("Some text content for my
        OffsetAttribute offsetAttribute = tokenStream.addAttribute(OffsetAttribute.class);
        tokenStream.reset();
        while (tokenStream.incrementToken()) {
            System.out.println("token: " + tokenStream.reflectAsString(true).toString());
            System.out.println("token start offset: " + offsetAttribute.startOffset());
            System.out.println("token end offset: " + offsetAttribute.endOffset());
        }
    }

    @Test
    public void testMultiPhraseQuery() throws IOException {
        Term[] terms = new Term[]{new Term("content", "has"), new Term("content", "lots")};
        Term term2 = new Term("content", "bridges");
        //多个add之间认为是OR操作，即(has lots)和bridges之间的slop不大于3，不计算标点
        MultiPhraseQuery multiPhraseQuery = new MultiPhraseQuery.Builder().add(terms).add(term2).setSlo
        IndexSearcher indexSearcher = new IndexSearcher(DirectoryReader.open(directory));
        TopDocs search = indexSearcher.search(multiPhraseQuery, 10);
        Assert.assertEquals(2, search.totalHits);
    }

    //使用BooleanQuery类模拟MultiPhraseQuery类的功能
    @Test
    public void testBooleanQueryImitateMultiPhraseQuery() throws IOException {
        PhraseQuery first = new PhraseQuery.Builder().setSlop(3).add(new Term("content", "Amsterdam")).a
                .build();
        PhraseQuery second = new PhraseQuery.Builder().setSlop(1).add(new Term("content", "Venice")).ad
        BooleanQuery booleanQuery = new BooleanQuery.Builder().add(first, BooleanClause.Occur.SHOULD).ad
        IndexSearcher indexSearcher = getIndexSearcher();
        TopDocs search = indexSearcher.search(booleanQuery, 10);
        Assert.assertEquals(2, search.totalHits);
    }
}
```

有关QueryParser的详细语法请参考这里。

Lucene的高级搜索技术

Lucene包含了一个建立在SpanQuery类基础上的整套查询体系，大致反映了Lucene的Query类体系。SpanQuery是指域中的起始语汇单元和终止语汇单元的位置。SpanQuery有一些常用的子类，如下所示

SpanQuery类型	描述

SpanQuery类型	描述
FieldMaskingSpanQuery	用于在多个域之间查询，即把另一个域看作某个域，从而看起来就像在同一个域里查询，因为 Lucene 默认某个条件只能作用在单个域上，不支持跨域查询，只能在同一个域里查询，所以有了 FieldMaskingSpanQuery
SpanTermQuery	和其它跨度查询类型结合使用，单独使用时相当于 TermQuery，唯一的区别就是使用 SpanTermQuery 可以得到 Term 的 Span 跨度信息
SpanNearQuery	用来匹配两个 Term 之间的跨度的，即一个 Term 经过几个跨度可以到达另一个 Term，slop 为跨度因子，用来限制两个 Term 之间的最大跨度。还有一个 inOrder 参数，它用来设置是否允许进行倒序跨度，什么意思？即 TermA 到 TermB 不一定是从左到右去匹配也可以从右到左，而从右到左就是倒序，inOrder 为 true 即表示 order（顺序）很重要不能倒序去匹配必须正向去匹配，false 则反之。注意停用词不在 slop 统计范围内
SpanFirstQuery	表示对出现在一个域中的[0, n]范围内的 term 项进行的匹配查询，关键是n指定了查询的 term 出现范围的上限
SpanContainingQuery	返回在另一个范围内的查询匹配结果，big 和 little 的子句可以是任何 span 类型查询。在包含 little 匹配中从 big 匹配跨度返回。例如“a beautiful and boring world”，big 查询是 SpanNearQuery(SpanTermQuery(“beautiful”), SpanTermQuery(“world”)).setSlop(2)，而 little 查询是 SpanTermQuery(“boring”)，则该 Doc 命中，并从 big 匹配跨度返回，即 big 优先级高
SpanWithinQuery	与 SpanContainingQuery 类似，只不过是从 little 匹配跨度返回，换句话说，SpanContainingQuery 是 big 查询优先级更高，而 SpanWithinQuery 是 little 查询优先级更高。也许英文说的更清晰，In clear, the SpanContainingQuery will keep matches that contain another Spans, while the SpanWithinQuery will keep matches that are contained within another Spans.
SpanNotQuery	使用场景是当使用 SpanNearQuery 时，如果两个 Term 从 TermA 到 TermB 有多种情况，即可能出现 TermA 或者 TermB 在索引中重复出现，则可能有多种情况，SpanNotQuery 就是用来限制 TermA 和 TermB 之间不存在 TermC，从而排除一些情况，实现更精确的控制
SpanOrQuery	这个查询会嵌套一些子查询，子查询之间的逻辑关系为或

使用示例

```
import org.apache.lucene.analysis.Analyzer;
import org.apache.lucene.analysis.TokenStream;
import org.apache.lucene.analysis.core.WhitespaceAnalyzer;
```

```
import org.apache.lucene.analysis.tokenattributes.OffsetAttribute;
import org.apache.lucene.document.Document;
import org.apache.lucene.document.Field;
import org.apache.lucene.document.StringField;
import org.apache.lucene.document.TextField;
import org.apache.lucene.index.*;
import org.apache.lucene.search.*;
import org.apache.lucene.search.spans.*;
import org.apache.lucene.store.RAMDirectory;
import org.junit.After;
import org.junit.Assert;
import org.junit.Before;
import org.junit.Test;

import java.io.IOException;
import java.io.StringReader;

public class SpanQueryDemo {
    private RAMDirectory directory;
    private IndexSearcher indexSearcher;
    private IndexReader indexReader;
    private SpanTermQuery quick;
    private SpanTermQuery brown;
    private SpanTermQuery red;
    private SpanTermQuery fox;
    private SpanTermQuery lazy;
    private SpanTermQuery sleepy;
    private SpanTermQuery dog;
    private SpanTermQuery cat;
    private Analyzer analyzer;
    private IndexWriter indexWriter;
    private IndexWriterConfig indexWriterConfig;

    @Before
    public void setUp() throws IOException {
        directory = new RAMDirectory();
        analyzer = new WhitespaceAnalyzer();
        indexWriterConfig = new IndexWriterConfig(analyzer);
        indexWriterConfig.setOpenMode(IndexWriterConfig.OpenMode.CREATE_OR_APPEND);
        indexWriter = new IndexWriter(directory, indexWriterConfig);
        Document document = new Document();
        //TextField使用Store.YES时索引文档、频率、位置信息
        document.add(new TextField("f", "What's amazing, the quick brown fox jumps over the lazy dog",
        indexWriter.addDocument(document);
        document = new Document();
        document.add(new TextField("f", "Wow! the quick red fox jumps over the sleepy cat", Field.Store
        indexWriter.addDocument(document);
        indexWriter.commit();
        indexSearcher = new IndexSearcher(DirectoryReader.open(directory));
        indexReader = indexSearcher.getIndexReader();
        quick = new SpanTermQuery(new Term("f", "quick"));
        brown = new SpanTermQuery(new Term("f", "brown"));
        red = new SpanTermQuery(new Term("f", "red"));
        fox = new SpanTermQuery(new Term("f", "fox"));
        lazy = new SpanTermQuery(new Term("f", "lazy"));
        dog = new SpanTermQuery(new Term("f", "dog"));
        sleepy = new SpanTermQuery(new Term("f", "sleepy"));
        cat = new SpanTermQuery(new Term("f", "cat"));
    }

    @After
    public void setDown() {
        if (indexWriter != null && indexWriter.isOpen()) {
            try {
                indexWriter.close();
```

```
            } catch (IOException e) {
                System.out.println(e);
            }
        }
    }

    private void assertOnlyBrownFox(Query query) throws IOException {
        TopDocs search = indexSearcher.search(query, 10);
        Assert.assertEquals(1, search.totalHits);
        Assert.assertEquals("wrong doc", 0, search.scoreDocs[0].doc);
    }

    private void assertBothFoxes(Query query) throws IOException {
        TopDocs search = indexSearcher.search(query, 10);
        Assert.assertEquals(2, search.totalHits);
    }

    private void assertNoMatches(Query query) throws IOException {
        TopDocs search = indexSearcher.search(query, 10);
        Assert.assertEquals(0, search.totalHits);
    }

    /**
     * 输出跨度查询的结果
     *
     * @param query
     * @throws IOException
     */
    private void dumpSpans(SpanQuery query) throws IOException {
        SpanWeight weight = query.createWeight(indexSearcher, true);
        Spans spans = weight.getSpans(indexReader.getContext().leaves().get(0), SpanWeight.Postings.POS
        System.out.println(query);
        TopDocs search = indexSearcher.search(query, 10);
        float[] scores = new float[2];
        for (ScoreDoc sd : search.scoreDocs) {
            scores[sd.doc] = sd.score;
        }
        int numSpans = 0;
        //处理所有跨度
        while (spans.nextDoc() != Spans.NO_MORE_DOCS) {
            while (spans.nextStartPosition() != Spans.NO_MORE_POSITIONS) {
                numSpans++;
                int id = spans.docID();
                //检索文档
                Document doc = indexReader.document(id);
                //重新分析文本
                TokenStream stream = analyzer.tokenStream("contents", new StringReader(doc.get("f")));
                stream.reset();
                OffsetAttribute offsetAttribute = stream.addAttribute(OffsetAttribute.class);
                int i = 0;
                StringBuilder sb = new StringBuilder();
                sb.append("  ");
                //处理所有语汇单元
                while (stream.incrementToken()) {
                    if (i == spans.startPosition()) {
                        sb.append("<");
                    }
                    sb.append(offsetAttribute.toString());
                    if (i + 1 == spans.endPosition()) {
                        sb.append(">");
                    }
                    sb.append(" ");
                    i++;
                }
                sb.append("(").append(scores[id]).append(")");
```

```
                System.out.println(sb.toString());
                stream.close();
            }
            if (numSpans == 0) {
                System.out.println(" No Spans");
            }
        }
    }

    @Test
    public void testSpanTermQuery() throws IOException {
        assertOnlyBrownFox(brown);
        dumpSpans(brown);
        dumpSpans(new SpanTermQuery(new Term("f", "the")));
        dumpSpans(new SpanTermQuery(new Term("f", "fox")));
    }

    /**
     * SpanFirstQuery可以对出现在域中前面某位置的跨度进行查询
     */
    @Test
    public void testSpanFirstQuery() throws IOException {
        SpanFirstQuery spanFirstQuery = new SpanFirstQuery(brown, 2);
        assertNoMatches(spanFirstQuery);
        dumpSpans(spanFirstQuery);
        //设置从前面开始10个跨度查找brown，一个切分结果是一个跨度，使用的是空格分词器
        spanFirstQuery = new SpanFirstQuery(brown, 5);
        dumpSpans(spanFirstQuery);
        assertOnlyBrownFox(spanFirstQuery);
    }

    @Test
    public void testSpanNearQuery() throws IOException {
        SpanQuery[] queries = new SpanQuery[]{quick, brown, dog};
        SpanNearQuery spanNearQuery = new SpanNearQuery(queries, 0, true);
        assertNoMatches(spanNearQuery);
        dumpSpans(spanNearQuery);
        spanNearQuery = new SpanNearQuery(queries, 4, true);
        assertNoMatches(spanNearQuery);
        dumpSpans(spanNearQuery);
        spanNearQuery = new SpanNearQuery(queries, 5, true);
        assertOnlyBrownFox(spanNearQuery);
        dumpSpans(spanNearQuery);
        //将inOrder设置为false，那么就不会考虑顺序，只要两者中间不超过三个就可以检索到
        spanNearQuery = new SpanNearQuery(new SpanQuery[]{lazy, fox}, 3, false);
        assertOnlyBrownFox(spanNearQuery);
        dumpSpans(spanNearQuery);
        //在考虑顺序的情况下，是检索不到的
        spanNearQuery = new SpanNearQuery(new SpanQuery[]{lazy, fox}, 3, true);
        assertNoMatches(spanNearQuery);
        dumpSpans(spanNearQuery);
        PhraseQuery phraseQuery = new PhraseQuery.Builder().add(new Term("f", "lazy")).add(new Term("f"
        assertNoMatches(phraseQuery);
        PhraseQuery phraseQuery1 = new PhraseQuery.Builder().setSlop(5).add(phraseQuery.getTerms()[0]).
        assertOnlyBrownFox(phraseQuery1);
    }

    @Test
    public void testSpanNotQuery() throws IOException {
        //设置slop为1
        SpanNearQuery quickFox = new SpanNearQuery(new SpanQuery[]{quick, fox}, 1, true);
        assertBothFoxes(quickFox);
        dumpSpans(quickFox);
        //第一个参数表示要包含的跨度对象，第二个参数则表示要排除的跨度对象
        SpanNotQuery quickFoxDog = new SpanNotQuery(quickFox, dog);
```

```
        assertBothFoxes(quickFoxDog);
        dumpSpans(quickFoxDog);
        //下面把red排除掉，那么就只能查到一条记录
        SpanNotQuery noQuickRedFox = new SpanNotQuery(quickFox, red);
        assertOnlyBrownFox(noQuickRedFox);
        dumpSpans(noQuickRedFox);
    }

    @Test
    public void testSpanOrQuery() throws IOException {
        SpanNearQuery quickFox = new SpanNearQuery(new SpanQuery[]{quick, fox}, 1, true);
        SpanNearQuery lazyDog = new SpanNearQuery(new SpanQuery[]{lazy, dog}, 0, true);
        SpanNearQuery sleepyCat = new SpanNearQuery(new SpanQuery[]{sleepy, cat}, 0, true);
        SpanNearQuery quickFoxNearLazyDog = new SpanNearQuery(new SpanQuery[]{quickFox, lazyDog}, 3, tr
        assertOnlyBrownFox(quickFoxNearLazyDog);
        dumpSpans(quickFoxNearLazyDog);
        SpanNearQuery quickFoxNearSleepyCat = new SpanNearQuery(new SpanQuery[]{quickFox, sleepyCat}, 3
        dumpSpans(quickFoxNearSleepyCat);
        SpanOrQuery or = new SpanOrQuery(new SpanQuery[]{quickFoxNearLazyDog, quickFoxNearSleepyCat});
        assertBothFoxes(or);
        dumpSpans(or);
    }

    /**
     * 测试安全过滤
     *
     * @throws IOException
     */
    @Test
    public void testSecurityFilter() throws IOException {
        Document document = new Document();
        document.add(new StringField("owner", "eric", Field.Store.YES));
        document.add(new TextField("keywords", "A B of eric", Field.Store.YES));
        indexWriter.addDocument(document);
        document = new Document();
        document.add(new TextField("owner", "jobs", Field.Store.YES));
        document.add(new TextField("keywords", "A B of jobs", Field.Store.YES));
        indexWriter.addDocument(document);
        document.add(new TextField("owner", "jack", Field.Store.YES));
        document.add(new TextField("keywords", "A B of jack", Field.Store.YES));
        indexWriter.addDocument(document);
        indexWriter.commit();
        TermQuery termQuery = new TermQuery(new Term("owner", "eric"));
        TermQuery termQuery1 = new TermQuery(new Term("keywords", "A"));
        //把FILTER看做是Like即可，也就是说owner必须是eric的才允许检索到
        Query query = new BooleanQuery.Builder().add(termQuery1, BooleanClause.Occur.MUST).add(termQuer
                BooleanClause.Occur.FILTER).build();
        System.out.println(query);
        indexSearcher = new IndexSearcher(DirectoryReader.open(directory));
        TopDocs search = indexSearcher.search(query, 10);
        Assert.assertEquals(1, search.totalHits);
        //如果不加安全过滤的话，那么应该检索到三条记录
        query = new BooleanQuery.Builder().add(termQuery1, BooleanClause.Occur.MUST).build();
        System.out.println(query);
        search = indexSearcher.search(query, 10);
        Assert.assertEquals(3, search.totalHits);
        String keywords = indexSearcher.doc(search.scoreDocs[0].doc).get("keywords");
        System.out.println("使用Filter查询: " + keywords);
        //使用BooleanQuery可以实现同样的功能
        BooleanQuery booleanQuery = new BooleanQuery.Builder().add(termQuery, BooleanClause.Occur.MUST)
        search = indexSearcher.search(booleanQuery, 10);
        Assert.assertEquals(1, search.totalHits);
        System.out.println("使用BooleanQuery查询: " + indexSearcher.doc(search.scoreDocs[0].doc).get("key
    }
}
```

禁用模糊查询和通配符查询

有时候，可能不希望用户使用模糊查询或者通配符查询，这时候可以通过实现自己的QueryParser达到禁用的目的。例如

```
import org.apache.lucene.analysis.Analyzer;
import org.apache.lucene.analysis.core.WhitespaceAnalyzer;
import org.apache.lucene.queryparser.classic.ParseException;
import org.apache.lucene.queryparser.classic.QueryParser;
import org.apache.lucene.search.Query;

/**
 * Description:禁用模糊查询和通配符查询，同样的如果希望禁用其它类型查询，只需要覆写对应的getXXXQuery方法即可
 * </P>
 *
 * @author Wang Xu
 * @version V1.0.0
 * @since V1.0.0
 * WebSite: http://codepub.cn
 * Licence: Apache v2 License
 */
public class CustomQueryParser extends QueryParser {
    public CustomQueryParser(String f, Analyzer a) {
        super(f, a);
    }

    @Override
    protected Query getFuzzyQuery(String field, String termStr, float minSimilarity) throws ParseException
        throw new ParseException("Fuzzy queries not allowed!");
    }

    @Override
    protected Query getWildcardQuery(String field, String termStr) throws ParseException {
        throw new ParseException("Wildcard queries not allowed!");
    }

    public static void main(String args[]) throws ParseException {
        CustomQueryParser customQueryParser = new CustomQueryParser("filed", new WhitespaceAnalyzer());
        customQueryParser.parse("a?t");
        customQueryParser.parse("junit~");
    }
}
```

多索引的搜索合并方法

如果存在多索引文件，需要如何搜索并合并搜索结果呢？此时需要使用MultiReader，通过该类的构造函数MultiReader(IndexReader... subReaders)可以接收多个IndexReader实例，而一个IndexReader实例对应一个索引文件目录，之后用该MultiReader实例初始化IndexSearcher。

注意，IndexReader实例是完全线程安全的，多线程可以调用其任何方法，而无需对IndexReader实例进行同步。如果你的应用需要外部同步操作，对你自己的对象进行同步，而不是Lucene的。

```
import org.apache.lucene.analysis.Analyzer;
import org.apache.lucene.analysis.core.WhitespaceAnalyzer;
import org.apache.lucene.document.Document;
import org.apache.lucene.document.Field;
import org.apache.lucene.document.StringField;
import org.apache.lucene.index.*;
import org.apache.lucene.search.IndexSearcher;
import org.apache.lucene.search.ScoreDoc;
import org.apache.lucene.search.TermRangeQuery;
import org.apache.lucene.search.TopDocs;
import org.apache.lucene.store.Directory;
```

```
import org.apache.lucene.store.RAMDirectory;
import org.apache.lucene.util.BytesRef;
import org.junit.After;
import org.junit.Assert;
import org.junit.Before;
import org.junit.Test;

import java.io.IOException;

/**
 * Description:测试有多个索引文件的情况下，如何进行搜索并合并搜索结果
 * </P>
 *
 * @author Wang Xu
 * @version V1.0.0
 * @since V1.0.0
 * WebSite: http://codepub.cn
 * Licence: Apache v2 License
 */
public class MultiReaderDemo {
    private Analyzer analyzer = new WhitespaceAnalyzer();
    Directory aDirectory = new RAMDirectory();
    Directory bDirectory = new RAMDirectory();
    IndexWriterConfig aIndexWriterConfig = new IndexWriterConfig(analyzer);
    IndexWriterConfig bIndexWriterConfig = new IndexWriterConfig(analyzer);
    IndexWriter aIndexWriter;
    IndexWriter bIndexWriter;

    @Before
    public void setUp() throws IOException {
        String[] animals = {"a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m", "n", "o", "
        aIndexWriter = new IndexWriter(aDirectory, aIndexWriterConfig);
        bIndexWriter = new IndexWriter(bDirectory, bIndexWriterConfig);
        for (int i = 0; i < animals.length; i++) {
            Document document = new Document();
            String animal = animals[i];
            document.add(new StringField("animal", animal, Field.Store.YES));
            if (animal.charAt(0) < 'n') {
                aIndexWriter.addDocument(document);
            } else {
                bIndexWriter.addDocument(document);
            }
        }
        aIndexWriter.commit();
        bIndexWriter.commit();
    }

    @After
    public void setDown() {
        try {
            if (aIndexWriter != null && aIndexWriter.isOpen()) {
                aIndexWriter.close();
            }
            if (bIndexWriter != null && bIndexWriter.isOpen()) {
                bIndexWriter.close();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    @Test
    public void testMultiReader() throws IOException {
        IndexReader aIndexReader = DirectoryReader.open(aDirectory);
        IndexReader bIndexReader = DirectoryReader.open(bDirectory);
```

```
        MultiReader multiReader = new MultiReader(aIndexReader, bIndexReader);
        IndexSearcher indexSearcher = new IndexSearcher(multiReader);
        TopDocs animal = indexSearcher.search(new TermRangeQuery("animal", new BytesRef("h"), new BytesR
        Assert.assertEquals(10, animal.totalHits);
        ScoreDoc[] scoreDocs = animal.scoreDocs;
        for (ScoreDoc sd : scoreDocs) {
            System.out.println(indexSearcher.doc(sd.doc));
        }
    }
}
```

输出结果如下

```
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<animal:h>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<animal:i>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<animal:j>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<animal:k>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<animal:l>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<animal:m>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<animal:n>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<animal:o>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<animal:p>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<animal:q>>
```

坚持原创技术分享，您的支持将鼓励我继续创作！

打赏

本文作者： Wang Xu

本文链接： http://www.codepub.cn/2016/05/24/Lucene-6-0-in-action-5-The-index-search-an-IndexSearcher/

版权声明： 本博客所有文章除特别声明外，均采用 CC BY-SA 4.0 许可协议。转载请注明出处！

Lucene

Lucene 6.0 实战（4）-文本分析器

读《把时间当作朋友》

分享到： 收藏夹 复制网址 邮件 微信 QQ空间 腾讯微博 豆瓣 更多 1